Fatec
Baixada Santista
Rubens Lara

Aula 10 — Splines e modelos aditivos

regression splines; smoothing splines; GAM (introdução)

Prof. Dr. João Paulo Ferreira de Mello

19 de outubro de 2026

Ciências de Dados
Fatec Rubens Lara – Baixada Santista

O que vamos ver hoje

- **De onde viemos:** na Aula 9, suavizar era prever em um ponto olhando só a vizinhança dele — KNN-regressão (*k* vizinhos mais próximos) e kernel. Ficaram três contas abertas: a média local *achata nas pontas*, não devolve *fórmula* e, com muitos preditores, a vizinhança *some*.
- **O novo:** as **splines** — polinômios por pedaços, emendados sem quina nos **nós**. Dois botões: escolher os nós (**splines de regressão**) ou pôr um nó em cada observação e frear a curvatura (**splines de suavização**). Somando uma spline por preditor, os **modelos aditivos generalizados** (GAM, *generalized additive models*).
- **Para onde vai:** a Aula 11 troca as curvas suaves por regras de decisão: as **árvores**.

O fio da aula: os **mesmos 150 imóveis** da Aula 9 — uma área, um preço em dois degraus — e, no fim, uma base com **três preditores**.

Lembrete: o placar da Aula 9, nestes mesmos 150 imóveis

Siglas que voltam hoje. MQO (mínimos quadrados ordinários, Aula 4): o ajuste do `lm()`, que minimiza a soma dos quadrados dos resíduos (RSS). **CV** (validação cruzada, Aula 7): $\mathrm{CV}(5)$ é o erro quadrático médio (MSE) de validação com 5 dobras — as *mesmas* dobras da Aula 9. **Distância até a verdade**: a raiz do erro quadrático médio (RMSE) entre a curva ajustada e f, nos 300 pontos da grade — só existe porque simulamos.

método	botão (escolhido pela CV)	$\mathrm{CV}(5)$	distância até a verdade
reta (MQO)	—	1 172	21,0
polinômio	grau 6	788	8,1
KNN-regressão	$k = 15$	785	7,5
kernel	$h = 12$	788	6,2

Em palavras: o piso é $25^2 = 625$, e os três flexíveis empatam (785 a 788). Mas cada um tem seu defeito: o **polinômio** é *global* — um só jogo de coeficientes para 40 e para 255 m² — e oscila nas pontas; a **média local** (KNN e kernel) não devolve fórmula e achata nas pontas. Hoje: o melhor dos dois — **polinômios** ajustados por MQO, mas **por pedaços**.

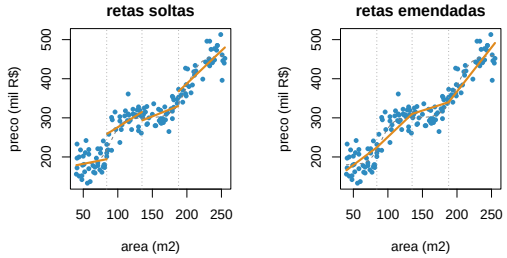
Splines de regressão

Cortar o eixo em pedaços: os nós

Em vez de um polinômio para a base inteira, um por pedaço. Os cortes são os **nós** (ξ, *knots*): aqui, os quartis da área — 84, 135 e 188 m², a escolha padrão do R, que não olha a verdade. Primeiro, retas:

```
nos    <- quantile(area, c(.25, .5, .75))     # os nos: 84, 135 e 188 m2
faixa  <- cut(area, c(-Inf, nos, Inf))        # o pedaco de cada imovel
soltas <- lm(preco ~ faixa * area)            # uma reta por pedaco
emend  <- lm(preco ~ bs(area, degree = 1, knots = nos))   # retas emendadas
fg <- cut(grade, c(-Inf, nos, Inf))           # o pedaco de cada ponto da grade
curvas <- list(
  "retas soltas"    = predict(soltas, data.frame(area = grade, faixa = fg)),
  "retas emendadas" = predict(emend,  data.frame(area = grade)))
par(mfrow = c(1,2), mar = c(4,4,2.2,1), pty = "s")
for (nome in names(curvas)) {
  plot(area, preco, pch = 16, col = cazul, cex = 0.8, main = nome,
       xlab = "area (m2)", ylab = "preco (mil R$)")
  abline(v = nos, lty = 3, col = "gray60")                  # os nos
  lines(grade, f(grade), col = "gray50", lty = 2)           # a verdade
  for (l in levels(fg))                                     # pedaco a pedaco
    lines(grade[fg == l], curvas[[nome]][fg == l], col = claranja, lwd = 2.5)
}
```

Soltas, as retas saltam; emendadas, fazem quina



Leia a figura: à esquerda, cada pedaço tem sua reta, ajustada só com os imóveis dele — e nos nós a curva *salta*: $+64$ mil R$ em 84 m², -22 em 135, $+37$ em 188 (imóveis de 84 e de 85 m² com 64 mil de diferença?). À direita, a regra "as retas se encontram no nó" deixa a curva contínua, mas com *quinas*: a inclinação pula de 1,32 para 1,68 mil R$ por m² em 84, de 1,68 para 0,57 em 135 e de 0,57 para 2,26 em 188.

Observação

Para sumir com as quinas: polinômios **cúbicos** por pedaço, emendados no valor, na inclinação *e* na curvatura. É a **spline cúbica**.

A spline cúbica: a emenda que não se vê

Definição

Spline cúbica com nós $\xi_1 < \xi_2 < \cdots < \xi_K$: uma função que, em cada pedaço (entre dois nós, antes do primeiro e depois do último), é um polinômio de grau 3 e que, em cada nó, é **contínua** e tem a **primeira e a segunda derivadas contínuas**.

Em palavras: dos dois lados de cada nó, batem o *valor*, a *inclinação* e a *curvatura* — a emenda não se vê. Só a terceira derivada muda no nó, e isso o olho não percebe.

Exemplo (quantos parâmetros, com $K = 3$ nós)

São 4 pedaços $\times$ 4 coeficientes $(a + bx + cx^2 + dx^3) = 16$. Cada nó impõe 3 restrições (valor, inclinação, curvatura): $3 \times 3 = 9$. Sobram $16 - 9 = \mathbf{7} = K + 4$. Nas retas emendadas da página anterior: $4 \times 2 = 8$ coeficientes, menos 1 restrição por nó (só o valor): $8 - 3 = 5 = K + 2$.

O número de parâmetros livres são os **graus de liberdade** (df, *degrees of freedom*) da spline — o botão de flexibilidade de hoje, no lugar do k e do h. Com K nós, a spline cúbica custa o mesmo que um polinômio de grau $K + 3$, mas é *local*: cada pedaço responde sobretudo aos dados do seu trecho.

Por dentro: uma base de funções e um MQO

Toda spline cúbica com nós $\xi_1, \ldots, \xi_K$ se escreve como

$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \sum_{k=1}^{K} \beta_{3+k}\,(x - \xi_k)_+^3, \qquad (x-\xi)_+^3 = \begin{cases} (x-\xi)^3, & x > \xi, \\ 0, & x \le \xi. \end{cases}$$

Em palavras: cada nó acrescenta *uma coluna* — zero antes do nó, um cubo que cresce depois dele. O cubo nasce com valor, inclinação e curvatura iguais a zero, e por isso somá-lo não quebra nada: a emenda é automática. Com as $K + 3$ colunas montadas, o ajuste é um **MQO** comum. É a **expansão de atributos** da Aula 8 — criar colunas a partir de x —, agora com cubos truncados. As colunas formam uma **base de funções**: toda spline com esses nós é uma soma delas.

```
entrada:  dados (x(i), y(i)), i = 1..n; nos xi(1) < ...  < xi(K)
1.  monte as colunas x, x^2, x^3 e, para cada no k, (x - xi(k))^3 se x > xi(k), senao 0
2.  ajuste y nessas K + 3 colunas por MQO, com intercepto:  K + 4 coeficientes
3.  para prever em x0, monte as mesmas colunas em x0 e some coeficiente vezes coluna
```

Na mão: a base truncada contra o bs( ) do R

```
cubo <- function(x, k) pmax(x - k, 0)^3     # (x - k)^3 depois do no; 0 antes
B <- cbind(area, area^2, area^3,
           cubo(area, nos[1]), cubo(area, nos[2]), cubo(area, nos[3]))
m_mao <- lm(preco ~ B)                       # MQO nas 6 colunas
m_bs  <- lm(preco ~ bs(area, knots = nos))   # a mesma spline, pelo R
c(length(coef(m_mao)), length(coef(m_bs)))   # quantos coeficientes
max(abs(fitted(m_mao) - fitted(m_bs)))       # os ajustes coincidem?
x0 <- c(1, 120, 120^2, 120^3, cubo(120, nos))  # as colunas em 120 m2
sum(coef(m_mao) * x0)                        # a predicao em 120 m2


#> [1] 7 7
#> [1] 3.99e-11
#> [1] 297.2
```

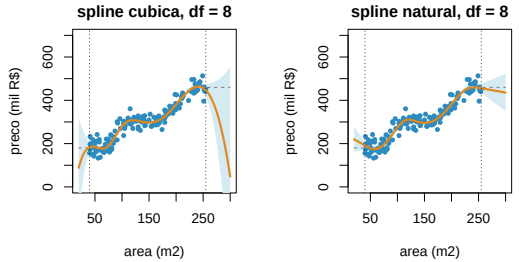
Leia a saída: os dois modelos têm 7 coeficientes ($K + 4$, com $K = 3$) e ajustes iguais até a 10ª casa decimal: colunas diferentes, mesmas curvas. Em 120 m² só o primeiro cubo está ligado — os nós de 135 e 188 "ainda não chegaram" — e a predição é 297,2; a verdade é $f(120) = 297{,}3$. O bs( ) usa outra base para as mesmas curvas (a *B-spline*), mais estável: as colunas truncadas chegam a $255^3 \approx 1{,}7 \times 10^7$.

Nas pontas há poucos dados: a spline natural

Antes do primeiro nó e depois do último, a cúbica fica solta, com poucos dados para segurá-la. A **spline natural** exige que, além dos **nós de fronteira** (os extremos dos dados), a curva seja uma **reta**. No R: `ns()` no lugar de `bs()`, com `df` graus de liberdade (o R põe os nós nos quantis). As duas com $\mathrm{df} = 8$, desenhadas até fora dos dados:

```
gx <- seq(20, 300, length = 300)              # a grade passa dos dados
ajustes <- list("spline cubica, df = 8"  = lm(preco ~ bs(area, df = 8)),
                "spline natural, df = 8" = lm(preco ~ ns(area, df = 8)))
par(mfrow = c(1,2), mar = c(4,4,2.2,1), pty = "s")
for (nome in names(ajustes)) {
  p <- predict(ajustes[[nome]], data.frame(area = gx), interval = "confidence")
  plot(area, preco, type = "n", xlim = c(20, 300), ylim = c(0, 700),
       main = nome, xlab = "area (m2)", ylab = "preco (mil R$)")
  polygon(c(gx, rev(gx)), c(p[, "lwr"], rev(p[, "upr"])),
          col = cfill, border = NA)                    # a faixa de 95%
  points(area, preco, pch = 16, col = cazul, cex = 0.8)
  lines(gx, f(gx), col = "gray50", lty = 2)            # a verdade
  lines(gx, p[, "fit"], col = claranja, lwd = 2.5)     # o ajuste
  abline(v = range(area), lty = 3)                     # onde os dados acabam
}
```

Fora dos dados, a cúbica despenca; a natural segue reta



Leia a figura: entre 40 e 255 m² as duas quase coincidem (distância até a verdade: 6,2 e 5,8). A diferença está nas pontas: a faixa de 95% da cúbica abre já perto dos extremos, e fora dos dados a curva despenca — prevê 48 mil R$ para 300 m² e 88 para 20 m² (o R até avisa do risco). A natural segue em reta: 436 e 220. A verdade: 460 e 180.

Observação

Com o mesmo $\mathrm{df} = 8$, a cúbica tem 5 nós internos e a natural, 7: a natural gasta menos nas pontas e investe no meio. Use `ns()` por padrão — mas nenhuma das duas *sabe* o que há além dos dados.

A promessa da Aula 9: a ponta sem achatar

Na reta pura da Aula 9 ($y = 100 + 2x$), o kernel previu as pontas "para dentro". Sorteamos o ruído 500 vezes: kernel ($h = 8$, a `nw( )` da Aula 9) contra spline natural ($\mathrm{df} = 4$), nas duas pontas.

```
pontas <- range(xb); set.seed(4)            # x = 1,5 e x = 99,3
sim <- replicate(500, {
  y <- 100 + 2*xb + rnorm(60, 0, 12)         # novo ruido, a mesma verdade
  c(kernel  = sapply(pontas, nw, x = xb, y = y, h = 8),
    natural = unname(predict(lm(y ~ ns(xb, df = 4)), data.frame(xb = pontas))))
})
round(rbind(media = rowMeans(sim), dp = apply(sim, 1, sd)), 1)


#>       kernel1 kernel2 natural1 natural2
#> media   118.8   282.0    102.8    298.6
#> dp        4.0     4.3      7.5      6.9
```

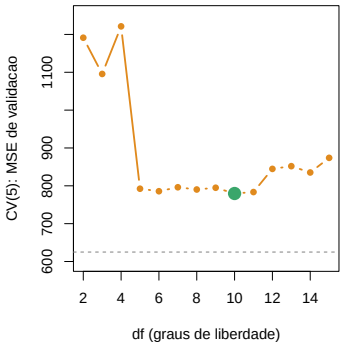
Leia a saída: a verdade nas pontas é 103,1 e 298,6. O kernel erra *sempre para dentro* ($+15{,}7$ e $-16{,}6$), com desvio padrão de 4: é **viés**. A spline acerta *na média* (102,8 e 298,6) e varia mais (7,5 e 6,9): sobra a **variância** de ter poucos dados na ponta. Ela não faz média de vizinhos — ajusta polinômios por MQO, que levam a tendência até a borda.

Quantos nós? Quem decide é a validação cruzada

```r
cv_ns <- function(d) mean(sapply(1:5, function(j) {
  tr <- dobra != j; va <- dobra == j        # as 5 dobras da Aula 9
  m  <- lm(preco ~ ns(area, df = d), data = dados[tr, ])
  mse(predict(m, dados[va, ]), preco[va])
}))
dfs  <- 2:15
cvns <- sapply(dfs, cv_ns)
setNames(round(cvns), dfs)                  # CV(5) por df

#>    2    3    4    5    6    7    8    9   10   11   12   13   14   15
#> 1191 1096 1221  792  786  796  790  795  780  784  845  852  835  874
```

```r
par(mar = c(4,4,1,1), pty = "s")
plot(dfs, cvns, type = "b", pch = 16, col = claranja, lwd = 2,
     ylim = c(600, 1250), xlab = "df (graus de liberdade)",
     ylab = "CV(5): MSE de validacao")
abline(h = 625, lty = 2, col = "gray50")                  # piso
points(10, cvns[dfs == 10], pch = 16, cex = 2, col = cverde)
```



Leia: o erro despenca de $\mathrm{df} = 4$ (1 221) para $\mathrm{df} = 5$ (792) e fica num platô até 11; o fundo é $\mathrm{df} = 10$ (**780**; o KNN fez 785). A curva é *serrilhada* porque os nós mudam de lugar a cada df: com 4, caem em 84, 135 e 188 m²; com 5, em 75, 117, 161 e 199 — um encosta no degrau de 200. A regra de um erro padrão (Aula 8) ficaria com $\mathrm{df} = 5$.

Splines de suavização

De onde vem o funcional: a régua flexível

Escolher os nós pesa (o U serrilhado). O outro caminho começa onde a palavra nasceu: *spline* era a **régua flexível** de desenhistas e construtores navais, presa por pesos nos pontos por onde a curva devia passar. Solta, ela assume a forma de **menor energia de flexão**, $\frac{EI}{2}\int \kappa(s)^2 ds \approx \frac{EI}{2}\int g''(x)^2 dx$ (κ é a curvatura e EI a rigidez da régua; a próxima página explica a conta).

Definição

Spline interpoladora (o funcional de origem). Dados (x_i, y_i), com $x_1 < \cdots < x_n$, é a função g que minimiza

$$\int_{x_1}^{x_n} g''(t)^2\, dt \qquad \text{sujeito a} \qquad g(x_i) = y_i, \quad i = 1, \ldots, n.$$

Em palavras: g'' mede quanto a inclinação muda, e o quadrado integrado soma a "dobra" da curva. Entre todas as curvas que passam pelos pontos, procuramos a que *menos dobra*.

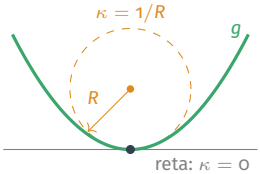
Observação

Resultado clássico (Holladay, 1957): a solução é a **spline cúbica natural com um nó em cada** x_i — cúbica porque, entre dois pesos, o cálculo de variações dá $g^{(4)} = 0$; natural porque, depois do último peso, nada obriga a régua a dobrar: ela segue reta.

A física da régua: curvatura e energia de flexão

Dobrar a régua estica as fibras de um lado e comprime as do outro. Num pedacinho ds, a energia guardada é $\frac{EI}{2}\kappa^2 ds$; na régua toda, $U = \frac{EI}{2}\int \kappa(s)^2 ds$.

- $\kappa = 1/R$ é a **curvatura**, o inverso do raio do círculo que melhor encosta na curva (figura): numa reta, $\kappa = 0$; numa dobra fechada, R é pequeno e κ é grande.
- E mede o material (aço $\gg$ madeira) e $I = bh^3/12$ a seção (largura b, espessura h): EI é a **rigidez à flexão**. Com o dobro da espessura, a régua fica 8 vezes mais rígida.



No gráfico de $y = g(x)$, $\kappa = g''/(1+g'^2)^{3/2}$ e $ds = \sqrt{1+g'^2}\,dx$. Então

$$U = \frac{EI}{2}\int \frac{g''(x)^2}{\left(1+g'(x)^2\right)^{5/2}}\,dx \;\approx\; \frac{EI}{2}\int g''(x)^2\,dx \qquad \text{quando a inclinação } |g'| \text{ é pequena.}$$

Exemplo (quanto vale o $\approx$)

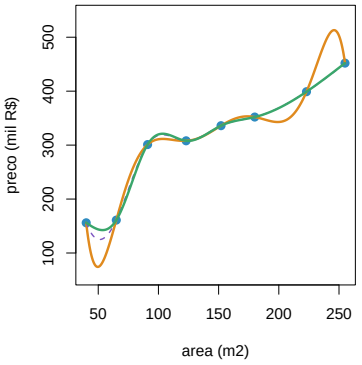
O fator $(1+g'^2)^{-5/2}$ vale 0,975 com $|g'| = 0{,}1$ (erro de 2,5%), 0,806 com 0,3 (19%) e 0,572 com 0,5 (43%). A estatística adota $\int g''^2$ mesmo assim: é quadrático em g, e o ajuste vira um MQO penalizado.

Pelos mesmos 8 pontos: quem dobra menos?

```r
i <- round(seq(1, n, length = 8)); xp <- area[i]; yp <- preco[i]  # 8 imoveis
regua <- splinefun(xp, yp, method = "natural") # a regua: spline natural
outra <- splinefun(xp, yp, method = "fmm")     # outra spline cubica
p7    <- lm(yp ~ poly(xp, 7))                  # grau 7: passa pelos 8
grau7 <- function(x) predict(p7, data.frame(xp = x))
tt <- seq(min(xp), max(xp), length = 20001); h <- tt[2] - tt[1]
flexao <- function(g)                          # integral de g''^2
  sum((diff(g(tt), differences = 2) / h^2)^2) * h
round(c(regua = flexao(regua), outra = flexao(outra), grau7 = flexao(grau7)), 2)

#> regua outra grau7
#>  4.36  6.94 85.12
```

```r
par(mar = c(4,4,1,1), pty = "s"); gg <- seq(min(xp), max(xp), length = 400)
plot(xp, yp, pch = 16, col = cazul, cex = 1.3, ylim = c(60, 540),
     xlab = "area (m2)", ylab = "preco (mil R$)")
lines(gg, grau7(gg), col = claranja, lwd = 2.5)          # polinomio de grau 7
lines(gg, outra(gg), col = croxo, lwd = 1.5, lty = 2)    # outra spline cubica
lines(gg, regua(gg), col = cverde, lwd = 2.5)            # a regua
```



Leia: as três curvas passam pelos 8 pontos. Energia de flexão: régua (verde) 4,36; outra spline cúbica, com outra regra nas pontas (roxa), 6,94; polinômio de grau 7 (laranja), 85,1 — quase vinte vezes mais, oscilando nas pontas. Nenhuma curva pelos 8 pontos dobra menos que a spline natural.

Com ruído: troque os pinos por molas

Presa nos pontos, a régua copia o ruído (sobreajuste). Com **molas** no lugar dos pinos, ela só precisa passar *perto*: $\min \int g''^2$ sujeito a $\sum_i (y_i - g(x_i))^2 \leq S$ (Reinsch, 1967). Penalizar equivale a limitar (Aula 8), e a restrição vira pedágio:

Definição

Spline de suavização (*smoothing spline*). Dado $\lambda \geq 0$, é a função g que minimiza

$$\underbrace{\sum_{i=1}^{n} (y_i - g(x_i))^2}_{\text{RSS: a energia das molas}} + \lambda \underbrace{\int g''(t)^2 \, dt}_{\text{a energia da régua}} .$$

Em palavras: cada observação puxa a régua com força proporcional à distância; λ é a rigidez da régua diante das molas (a física, na próxima página).

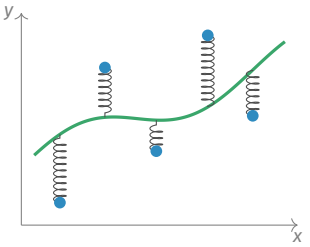
- $\lambda \to 0$: a régua volta a passar pelos pontos (a interpoladora); $\lambda \to \infty$: régua rígida, a **reta de MQO**.
- A solução segue sendo uma **spline cúbica natural com nó em cada x_i distinto**, encolhida pelo λ, como no Ridge.

A física do outro funcional: a régua presa por molas

Troque cada pino por uma **mola** de rigidez k entre o ponto (x_i, y_i) e a régua. Esticada de $d = y_i - g(x_i)$ (o resíduo), a mola guarda $\frac{k}{2}d^2$. A energia total é

$$U(g) = \underbrace{\frac{k}{2}\sum_{i=1}^{n}\big(y_i - g(x_i)\big)^2}_{\text{as molas}} + \underbrace{\frac{EI}{2}\int g''(x)^2\,dx}_{\text{a régua}}.$$

Dividir por $k/2$ não muda onde fica o mínimo, e sobra $\sum_i \big(y_i - g(x_i)\big)^2 + \frac{EI}{k}\int g''^2$: o funcional da spline de suavização, com $\boldsymbol{\lambda = EI/k}$.



verde: a régua; cinza: as molas; azul: as observações

Em palavras: λ é a rigidez da régua dividida pela das molas. Régua mole ($\lambda \to 0$): as molas a levam aos pontos (a interpoladora). Régua rígida ($\lambda \to \infty$): sobra a reta de menor RSS, a **reta de MQO**.

Observação

No equilíbrio, forças e momentos das molas se anulam: com $r_i = y_i - g(x_i)$, $\sum_i r_i = 0$ e $\sum_i r_i x_i = 0$, as equações normais da reta de MQO (com $\mathrm{df} = 10$: `sum(r)` $\approx 3 \times 10^{-9}$ e `sum(r * area)` $\approx 1 \times 10^{-7}$).

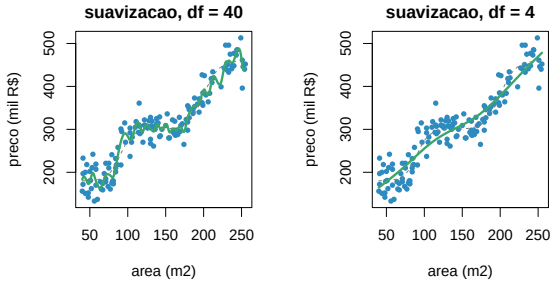
O botão λ, medido em graus de liberdade

O valor de λ é difícil de ler (depende da escala de x). Por isso o R aceita o botão em **graus de liberdade efetivos**, df_λ: de 2 ($\lambda \to \infty$, a reta) até o número de valores distintos de x ($\lambda = 0$; aqui, 110 — o `smooth.spline`, por economia, usa menos nós e para em 67). Com $\mathrm{df} = 40$ e com $\mathrm{df} = 4$:

```
par(mfrow = c(1,2), mar = c(4,4,2.2,1), pty = "s")
for (d in c(40, 4)) {
  s <- smooth.spline(area, preco, df = d)       # o R acha o lambda desse df
  plot(area, preco, pch = 16, col = cazul, cex = 0.8,
       main = paste("suavizacao, df =", d),
       xlab = "area (m2)", ylab = "preco (mil R$)")
  lines(grade, f(grade), col = "gray50", lty = 2)        # a verdade
  lines(predict(s, grade), col = cverde, lwd = 2.5)      # a spline
}
```

Em palavras: o ajuste é linear nos y_i ($\hat{\mathbf{y}} = \mathbf{S}_\lambda \mathbf{y}$), e df_λ soma, em todas as observações, o peso que y_i tem no próprio $\hat{y}_i$ (o traço de $\mathbf{S}_\lambda$). Num MQO com 7 coeficientes, essa soma dá exatamente 7: os graus de liberdade efetivos generalizam "o número de parâmetros".

df grande treme; df pequeno achata



Leia a figura: com $\mathrm{df} = 40$ (freio fraco) a curva treme — segue o ruído (variância alta). Com $\mathrm{df} = 4$ (freio forte) os degraus viram rampa (viés alto). Distância até a verdade: 14,1 e 15,8.

Observação

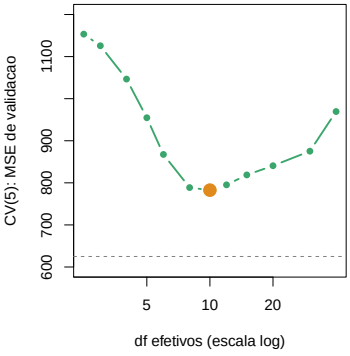
É o dilema do *k* e do *h* da Aula 9 — mas aqui, como no grau do polinômio, **df maior é mais flexível**. Em termos de λ, o contrário: λ maior, curva mais rígida.

Escolhendo o λ (o df) por validação cruzada

```r
cv_ss <- function(d) mean(sapply(1:5, function(j) {
  tr <- dobra != j; va <- dobra == j          # as mesmas dobras
  s  <- smooth.spline(area[tr], preco[tr], df = d)
  mse(predict(s, area[va])$y, preco[va])
}))
dfss <- c(2.5, 3, 4, 5, 6, 8, 10, 12, 15, 20, 30, 40)
cvss <- sapply(dfss, cv_ss)
setNames(round(cvss), dfss)                   # CV(5) por df
smooth.spline(area, preco)$df                 # sem df: o R escolhe (GCV)

#>  2.5    3    4    5    6    8   10   12   15   20   30   40
#> 1153 1126 1047  955  867  789  783  795  819  841  875  969
#> [1] 10.06
```

```r
par(mar = c(4,4,1,1), pty = "s")
plot(dfss, cvss, type = "b", log = "x", pch = 16, col = cverde, lwd = 2,
     ylim = c(600, 1200), xlab = "df efetivos (escala log)",
     ylab = "CV(5): MSE de validacao")
abline(h = 625, lty = 2, col = "gray50")                  # piso
points(10, cvss[dfss == 10], pch = 16, cex = 2, col = claranja)
```



Leia: agora o U é *liso* — não há nós mudando de lugar, só o freio. O fundo é $\mathrm{df} = 10$ (**783**). Sem o argumento `df`, o `smooth.spline` escolhe pela **validação cruzada generalizada** (GCV), um atalho que não reajusta o modelo dobra a dobra: $\mathrm{df} = 10{,}06$, o mesmo lugar.

O placar com um preditor: empate — e o que as splines trazem

método	botão (pela CV)	$\mathrm{CV}(5)$	distância até a verdade
reta (MQO)	—	1 172	21,0
polinômio	grau 6	788	8,1
KNN-regressão	$k = 15$	785	7,5
kernel	$h = 12$	788	6,2
spline natural	$\mathrm{df} = 10$	**780**	6,7
spline de suavização	$\mathrm{df} = 10$	783	**5,5**

Em palavras: em validação cruzada, os flexíveis empatam de novo (780 a 788, com piso de 625). A distância até a verdade favorece a spline de suavização, por pouco — e fora da simulação ninguém a vê. O ganho das splines está em outro lugar:

- **são um MQO:** a spline de regressão é um `lm()` comum — fórmula, `predict()`, faixa de confiança, tudo vale;
- **nas pontas:** sem o viés da média local; e a natural vira reta fora dos dados;
- **somam-se:** uma spline por preditor, somadas, ainda é um MQO (penalizado). É a saída para a maldição: o GAM.

Modelos aditivos (GAM)

Muitos preditores: uma curva para cada um, somadas

Na Aula 9, a maldição: com muitos preditores, *vizinho* deixa de existir. A regressão múltipla da Aula 4 escapava dela somando um termo por preditor — só que reto. O GAM mantém a soma e deixa cada termo entortar:

Definição

Modelo aditivo generalizado (GAM, *generalized additive model*):

$$y_i = \beta_0 + f_1(x_{i1}) + f_2(x_{i2}) + \cdots + f_p(x_{ip}) + \varepsilon_i,$$

com cada f_j uma função suave (uma spline) de *um só* preditor.

Em palavras: cada preditor ganha a sua curva, e a predição é a soma das curvas. Cada f_j vive em *uma* dimensão — onde, como vimos, suavizar funciona. A regressão múltipla é o caso em que toda curva é uma reta, $f_j(x) = \beta_j x$.

- **Generalizado:** o mesmo vale para classificação, somando as curvas no logaritmo da chance — a logística da Aula 5 (`family = binomial`).
- **O preço:** *sem interações*. O efeito da área é o mesmo perto e longe da praia; se não for, as curvas somadas não veem (a não ser que você peça — no fim da seção).

Como o gam( ) ajusta: a base da seção 1 com o freio da seção 2

```
entrada:  dados (x(i,1), ..., x(i,p), y(i)), i = 1..n
1.  para cada preditor j que leva curva, monte uma base de spline (umas 10 colunas)
2.  junte todas as colunas numa matriz so — uma expansao de atributos (Aula 8)
3.  ajuste por MQO penalizado:  RSS + lambda(1) curv(f1) + ...  + lambda(p) curv(fp),
    com curv(f) = integral de f''(t)^2 — um freio por preditor
4.  escolha os lambdas por GCV (o gam() faz isso sozinho)
5.  devolva beta0, as curvas f1, ..., fp e os graus de liberdade efetivos de cada uma
```

Em palavras: nada de novo — só montado junto. A base vem das splines de regressão, o freio das de suavização, e a matriz grande com penalidade é a Aula 8. Cada curva sai com seus **graus de liberdade efetivos** (edf, *effective degrees of freedom*): $\mathrm{edf} \approx 1$ é uma reta; quanto maior, mais a curva dobra.

- No R: pacote `mgcv` (vem na instalação padrão), `gam(y ~ s(x1) + s(x2) + x3)`. O `s()` pede curva; sem ele, o preditor entra reto — é assim que entram os qualitativos.
- Dá para fazer um GAM "à mão": `lm(y ~ ns(x1, df = 8) + ns(x2, df = 4) + x3)` é um MQO comum — mas aí os df são escolhidos por você, um a um.

Três preditores e uma verdade aditiva

Uma base maior da mesma imobiliária: 300 imóveis, com área, **distância até a praia** (km) e **idade** (anos). Simulada, para sabermos a verdade:

$$\text{preço} = 220 + \underbrace{f_a(\text{área})}_{\text{os degraus da Aula 9}} + \underbrace{150\, e^{-\text{dist}/0{,}8}}_{\text{o prêmio da praia}} - \underbrace{1{,}5 \cdot \text{idade}}_{\text{uma reta}} + \varepsilon, \qquad \varepsilon \sim N(0, 25^2).$$

Em palavras: f_a são os dois degraus da Aula 9 (sem o 180); o prêmio da praia cai à metade a cada $0{,}8 \ln 2 \approx 0{,}55$ km; a idade tira 1,5 mil R$ por ano, em linha reta. O `gam( )` não sabe disso:

```
g <- gam(preco ~ s(area) + s(dist) + s(idade), data = imov)
round(summary(g)$s.table[, "edf"], 2)     # graus de liberdade efetivos
g$sig2                                    # estimativa de sigma^2 (verdade: 625)


#>  s(area)  s(dist) s(idade)
#>     8.27     4.12     1.00
#> [1] 679.4
```

Leia a saída: 8,27 graus de liberdade na área (os degraus), 4,12 na distância (a curva da praia) e **1,00** na idade — o `gam( )` decidiu sozinho que a idade é uma reta. A variância do ruído estimada, 679, está perto da verdadeira, 625.

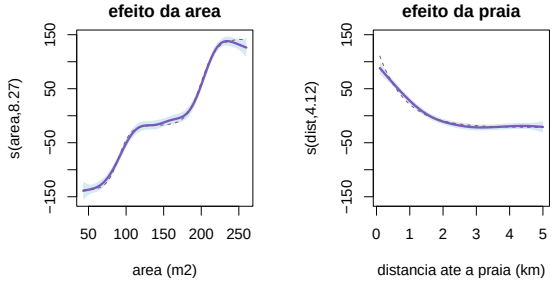
As curvas que o GAM achou, contra a verdade

`plot(g)` desenha cada $\hat{f}_j$ com uma faixa de ± 2 erros padrão. As curvas do GAM são *centradas* (média zero nos dados) — por isso centramos também a verdade, tracejada (`fa` e `fd` são as partes dela, criadas junto com os dados):

```
par(mfrow = c(1,2), mar = c(4,4,2.2,1), pty = "s")
plot(g, select = 1, scheme = 1, shade.col = cfill, col = croxo, lwd = 2.5,
     rug = FALSE, main = "efeito da area", xlab = "area (m2)")
curve(fa(x) - mean(fa(imov$area)), add = TRUE, lty = 2, col = "gray40")
plot(g, select = 2, scheme = 1, shade.col = cfill, col = croxo, lwd = 2.5,
     rug = FALSE, main = "efeito da praia", xlab = "distancia ate a praia (km)")
curve(fd(x) - mean(fd(imov$dist)), add = TRUE, lty = 2, col = "gray40")
```

O rótulo do eixo vertical, `s(area,8.27)`, já traz o edf. A idade fica fora da figura: com $\mathrm{edf} = 1$, a curva dela é uma reta.

Cada curva se lê com os outros preditores fixos



Leia a figura: a curva da área refaz os dois degraus; a da praia refaz a queda. O eixo vertical é o *efeito*, em mil R$, com os outros preditores fixos; o zero é o imóvel "médio". De 50 a 250 m² a área move o preço em 269 mil (a verdade: 278); de 100 m a 5 km da praia, a distância o move em 109 (a verdade: 132) — perto da praia, onde a queda é forte e o freio cobra pela curvatura, a curva fica um pouco abaixo. A idade: uma reta de $-1{,}68$ mil R$ por ano (a verdade: $-1{,}5$).

Prever é somar as curvas

Um imóvel de 120 m², a 500 m da praia, com 10 anos — e o mesmo a 2 km:

```
novos <- data.frame(area = 120, dist = c(0.5, 2), idade = 10)
predict(g, novos, type = "terms")          # o valor de cada curva
predict(g, novos)                          # beta0 + a soma das curvas


#>   s(area) s(dist) s(idade)
#> 1  -19.78   60.09    26.08
#> 2  -19.78  -10.88    26.08
#> attr(,"constant")
#> (Intercept)
#>       339.9
#>     1     2
#> 406.3 335.4
```

Leia a saída: $\hat{\beta}_0 = 339{,}9$ é o preço médio. A 500 m: $339{,}9 - 19{,}8 + 60{,}1 + 26{,}1 = 406{,}3$ mil (verdade: 402,6). A 2 km, *só a parte da praia muda* — de $+60{,}1$ para $-10{,}9$ —, e o preço cai 71 mil: "com os outros fixos" é a própria conta. Já "o efeito de 1 m²" não existe: é a inclinação da curva, uns 0,3 mil R$ por m² no patamar de 120 a 170 m² e de 3 a 4 nos degraus; a idade, reta, tem um só ($-1{,}68$ por ano).

O placar com três preditores — e onde o GAM falha

```
cv3 <- function(ajusta) mean(sapply(1:5, function(j) {   # 5 dobras, as mesmas
  tr <- imov[dobra3 != j, ]; va <- imov[dobra3 == j, ]   # para todos os modelos
  mse(ajusta(tr, va), va$preco)
}))
cv3(function(tr, va) predict(lm(preco ~ area + dist + idade, data = tr), va))
cv3(function(tr, va) predict(gam(preco ~ s(area) + s(dist) + s(idade), data = tr), va))


#> [1] 1383
#> [1] 723
```

Leia a saída: o MQO com os preditores retos fica em 1 383; o GAM, em **723**, perto do piso de 625 (prever a média dá 10 399). O KNN com os três preditores padronizados, no melhor k ($k = 8$), fez 1 357 — o mesmo que o MQO: a maldição já morde com $p = 3$.

Observação

O GAM ganhou porque **a verdade é aditiva**. Se o m² valesse mais na orla (uma *interação*), as curvas somadas não veriam; `+ ti(area, dist)` soma uma superfície de interação. Aqui ela piora a CV (739): não há interação a achar.

Exercícios e trabalho

Exercícios — cálculo

Exercício

Spline cúbica com $K = 4$ nós: quantos coeficientes têm os 5 polinômios soltos, quantas restrições os nós impõem e quantos parâmetros sobram? E a spline natural com esses 4 nós internos e os 2 de fronteira?

Exercício

Seja $f(x) = 100 + 2x + 3(x-50)_+ - 4(x-80)_+$, com $(u)_+ = \max(u, 0)$. Calcule $f(40)$, $f(50)$, $f(60)$ e $f(100)$ e a inclinação em cada pedaço. A curva é contínua em $x = 50$?

Exercício

Em $[0, 1]$, calcule o pedágio $\int_0^1 g''(t)^2 dt$ de $g_1(x) = 3 + 2x$, $g_2(x) = x^2$ e $g_3(x) = x^3$. Três curvas candidatas têm RSS 52, 30 e 12 e pedágios 0, 4 e 30: qual minimiza o critério com $\lambda = 0{,}1$, com $\lambda = 1$ e com $\lambda = 10$?

Exercício

Um GAM deu $\hat{\beta}_0 = 340$ e, para um imóvel, $\hat{f}_{\text{área}} = -20$, $\hat{f}_{\text{praia}} = +60$ e $\hat{f}_{\text{idade}} = +26$. Qual é a predição? Se o imóvel estivesse a 2 km, com $\hat{f}_{\text{praia}} = -11$, quanto ela mudaria — e por que só um termo muda?

Exercícios — conceituais

Exercício (Global $\times$ por pedaços)

Por que um polinômio de grau 10 oscila nas pontas e uma spline natural com 10 graus de liberdade não?

Exercício (Os dois botões)

Qual é o botão da spline de regressão e qual o da spline de suavização? Para que lado cada um aumenta a flexibilidade? Por que a curva de validação cruzada da primeira saiu serrilhada e a da segunda, lisa?

Exercício (Aditivo)

O que o GAM supõe sobre os preditores? Dê um exemplo imobiliário em que essa hipótese falha e diga o que o GAM erraria.

Exercício (O efeito de 1 m^2)

A diretoria quer "o efeito de 1 m^2 a mais no preço". O GAM da aula responde? E "o efeito de 1 ano a mais de idade"?

Laboratório — no seu banco de dados

Use a resposta e os preditores numéricos da **ficha da sua pergunta** (Entrega 1): troque `y`, `x1`, `x2` e `x3` pelas colunas do seu banco. MQO, spline e GAM, pelas mesmas dobras:

```r
library(splines); library(mgcv)
d <- na.omit(data.frame(y = dados$y, x1 = dados$x1, x2 = dados$x2, x3 = dados$x3))
set.seed(1); dobra <- sample(rep(1:5, length = nrow(d)))
cv_de <- function(ajusta) mean(sapply(1:5, function(j) {        # CV(5) de um modelo
  tr <- d[dobra != j, ]; va <- d[dobra == j, ]
  mean((va$y - ajusta(tr, va))^2)
}))
cv_de(function(tr, va) mean(tr$y))                              # linha de base
cv_de(function(tr, va) predict(lm(y ~ x1 + x2 + x3, data = tr), va))  # MQO
sapply(2:8, function(k) cv_de(function(tr, va)                  # spline em x1
  predict(lm(y ~ ns(x1, df = k) + x2 + x3, data = tr), va)))
cv_de(function(tr, va) predict(gam(y ~ s(x1) + s(x2) + s(x3), data = tr), va))  # GAM
g <- gam(y ~ s(x1) + s(x2) + s(x3), data = d)
round(summary(g)$s.table[, "edf"], 2)                           # perto de 1 = reta
par(mfrow = c(1, 3), pty = "s"); plot(g, scheme = 1)            # as curvas
```

Entregar (código em `estrutura/codigo/`, registro em `consolidados/`): o erro de CV da linha de base, do MQO, da melhor spline e do GAM; os edf e a figura das curvas; *uma frase* dizendo que preditor não é uma reta, e onde; e **o que isso muda ou indica para a sua pergunta** — o GAM bate a linha de base da ficha (se ela usa RMSE, tire a raiz)? Alguma curva pede rever um preditor? Resposta binária: veja a lista.

Resumo

- **Spline:** polinômios por pedaços, emendados nos **nós** sem quina. Por dentro, um **MQO numa base de funções** — a expansão da Aula 8.
- **De regressão:** o botão é o número de nós (o df), escolhido por validação cruzada; a **natural** vira reta nas pontas — use `ns( )`.
- **De suavização:** a régua de menor dobra ($\int g''^2$) presa por molas (o RSS); o botão λ vira df efetivos, e a CV (ou a GCV) escolhe.
- **GAM:** uma curva por preditor, somadas — escapa da maldição supondo *aditividade*; edf ≈ 1 é reta.

Próxima aula: de funções suaves a **regras de decisão** — as **árvores**: em vez de curvas somadas, perguntas em sequência ("a área passa de 150 m²? a praia fica a menos de 1 km?"), que pegam interações que o GAM não vê.